

HÁZI FELADAT

Programozás alapjai 2.

Feladatválasztás/feladatspecifikáció

Kálmán Patrik

XG5YQ1

2025. április 11.

TARTALOM

1. Feladat	4
2. Feladatspecifikáció.....	4
3. Terv	5
3.1 Objektum terv.....	5
3.2 Algoritmusok.....	9
4. Megvalósítás.....	12
4.1 Osztályok bemutatása.....	12
4.1.1 String osztályreferencia.....	13
4.1.2 Nev osztályreferencia.....	17
4.1.3 Cim osztályreferencia.....	20
4.1.4 Tel osztályreferencia	23
4.1.5 Profil osztályreferencia	26
4.1.6 Telefonkönyv osztályreferencia	28
4.2 Forráskód és dokumentáció.....	31
4.3 Menüvezérelt felhasználói felület	32
4.3 Hibakezelés	33
5. Tesztelés.....	35
5.1 teszt_1 ().....	37
5.2 teszt_2 ().....	38
5.3 teszt_3 ().....	38
5.4 teszt_4 ().....	39
5.5 teszt_5 ().....	39
5.6 Memóriakezelés tesztje	40
6. Mellékletek.....	40
6.1 string.h.....	40
6.2 string.cpp	41
6.3 nev.h.....	42
6.4 nev.cpp	43
6.5 cim.h.....	44
6.6 cim.cpp	45
6.7 tel.h.....	46
6.8 tel.cpp	47
6.9 profil.h.....	48
6.10 profil.cpp	48
6.11 telefonkönyv.h.....	49
6.12 telefonkönyv.cpp	50
6.13 teszt.h.....	51
6.14 teszt.cpp.....	52

6.15 menu.h	55
6.16 menu.cpp	55
6.17 main.cpp	57
7. Források.....	59

1. Feladat

Telefonkönyv!

Tervezze meg egy telefonkönyv alkalmazás egyszerűsített objektummodelljét, majd valósítsa azt meg! A telefonkönyvben az alábbi adatokat akarjuk tárolni:

- név (vezetéknév, keresztnév)
- becenév
- cím
- munkahelyi szám
- privát szám

Az alkalmazással minimum a következő műveleteket kívánjuk elvégezni:

- adatok felvétele
- listázás
- adatok törlése
- egyszerű keresés

A rendszer lehet bővebb funkcionálisú ezért nagyon fontos, hogy jól határozom meg az objektumokat. Demonstrálom a működést külön modulként tesztprogrammal! A megoldáshoz **nem** használok STL tárolót!

2. Feladatspecifikáció

A feladat egy egyszerű elektronikus telefonkönyv alkalmazás megtervezése és megvalósítása objektumorientált rendszer alapján. Mivel a telefonkönyvben tetszőleges számú bejegyzést kívánunk tárolni, az adatokat dinamikus memóriakezeléssel kezeljük.

Az adatok szöveg és szám formátumúak. Az alkalmazás lehetővé teszi a felhasználó számára, hogy korlátlan mennyiségű adatot rögzítsen.

Az alkalmazás a felhasználó számára az adatok felvételét, listázását, törlését, valamint a meglévő adatok keresését biztosítja. Az alkalmazás program megoldása további bővítésre is alkalmas, ezért a fejlesztés során kiemelt figyelmet fordítunk az objektumok megfelelő struktúrájára.

A program kezeli a hibákat. Amennyiben egy művelet sikertelen, a rendszer megfelelő visszajelzést ad a felhasználó számára.

A program tesztelésére egy külön modult készítek, amely ellenőrzi a funkciók helyes működését a közölt adatokkal, valamint a tetszőleges a más által adott adatokkal is.

3. Terv

A feladat egy telefonkönyv rendszer megtervezését, valamint egy tesztprogram megtervezését igényli.

3.1 Objektum terv

A telefonkönyv alkalmazást több osztályból álló objektumrendszerrel valósítom meg. Az osztálystruktúra a következőképpen épül fel:

String osztály:

A String osztály egy egyszerű implementációja a C-sztringek (nullával lezárt karakterláncok) kezelésére. Az osztály támogatja a dinamikus memória kezelést, másoló konstruktort, operátorokat és egyéb fontos műveleteket.

Adattagjai:

- **pData:** pointer az adatra (char* típusú)
- **len:** hossz, lezáró nulla nélkül (size_t típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- **printDbg():** Hibakeresési információ kiírása
- **size():** A string hosszának lekérdezése
- **Operátor túlterhelések** a különböző műveletek elvégzéséhez
- **c_str():** C sztringet ad vissza

Nev osztály:

A Nev osztály tárolja és kezeli egy személy nevét (vezetéknév, keresztnév, becenév). A név különböző formáit (pl. teljes név) visszaadhatjuk, és az osztály tartalmaz konstruktorokat, hogy a nevet különböző módokon inicializáljuk.

Adattagjai:

- **vnev:** Vezetéknév (String típusú)
- **knev:** Keresztnév (String típusú)
- **bnev:** Becenév (String típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (**getVnev()**, **getKnev()**, **getBnev()**)
- **getTeljesNev():** A teljes név összeállítása
- **Operátor túlterhelés** a kiíratáshoz (**operator<<**)
- **Beolvas_nev()** metódus

Cím osztály:

A Cím osztály tárolja és kezeli egy cím adatait, beleértve az országot, irányítószámot, várost és utcát. Az osztály különböző konstruktorokat tartalmaz, és lehetővé teszi a címek különböző formájú beolvasását és kiírását.

Adattagjai:

- **ország:** Ország megnevezése (String típusú)
- **iranyitoszam:** Irányítószám (String típusú)
- **varos:** Város neve (String típusú)
- **utca:** Utca neve (String típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (`getOrszag()`, `getIrandito()`, `getVaros()`, `getUtca()`)
- Operátor túlterhelés a kiíratáshoz (`operator<<`)
- `Beolvas_cim()` metódus

Tel osztály:

A Tel osztály tárolja és kezeli egy személy telefonszámadatait, beleértve a munkahelyi és a privát telefonszámot. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve a telefonszámok kezelését, beolvasását és kiírását.

Adattagjai:

- **mszam:** Munkahelyi szám (String típusú)
- **pszam:** Privát szám (String típusú)

Metódusai:

- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (`getMszam()`, `getPszam()`,)
- Operátor túlterhelés a kiíratáshoz (`operator<<`)
- `Beolvas_tel()` metódus

Telefonkönyv osztály:

A Telefonkönyv osztály tárolja és kezeli a profilok listáját egy telefonkönyvben. Az osztály lehetővé teszi a profilok hozzáadását, törlését, keresését és listázását. Az adatok dinamikusan tárolódnak, és az osztály automatikusan kezeli a kapacitás növelését.

Adattagjai:

- **profiltar:** Profil típusú objektumokat tároló pointer
- **profilSzam:** A jelenleg tárolt profilok száma (size_t típusú)
- **kapacitas:** A telefonkönyv maximális kapacitása (size_t típusú)

Fontosabb metódusai:

- **Konstruktor** és **destruktor** a megfelelő memóriakezeléshez
- **addProfil():** Új profil hozzáadása a telefonkönyvhöz
- **torol():** Profil törlése megadott paraméter alapján
- **keres metódusok:** Különböző keresési lehetőségek (**név**, **cím**, **telefonszám** szerint)
- **noveloKapacitas():** A telefonkönyv kapacitásának dinamikus növelése
- **getProfilSzam():** Visszaadja a tárolt profilok számát

Profil osztály:

A Profil osztály tárolja és kezeli egy személy adatprofilját, beleértve a nevét, címét és telefonszámát. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve az adatok kezelését, beolvasását és kiírását.

Adattagjai:

- **nev:** Nev típusú objektum
- **cim:** Cim típusú objektum
- **tel:** Tel típusú objektum

Metódusai:

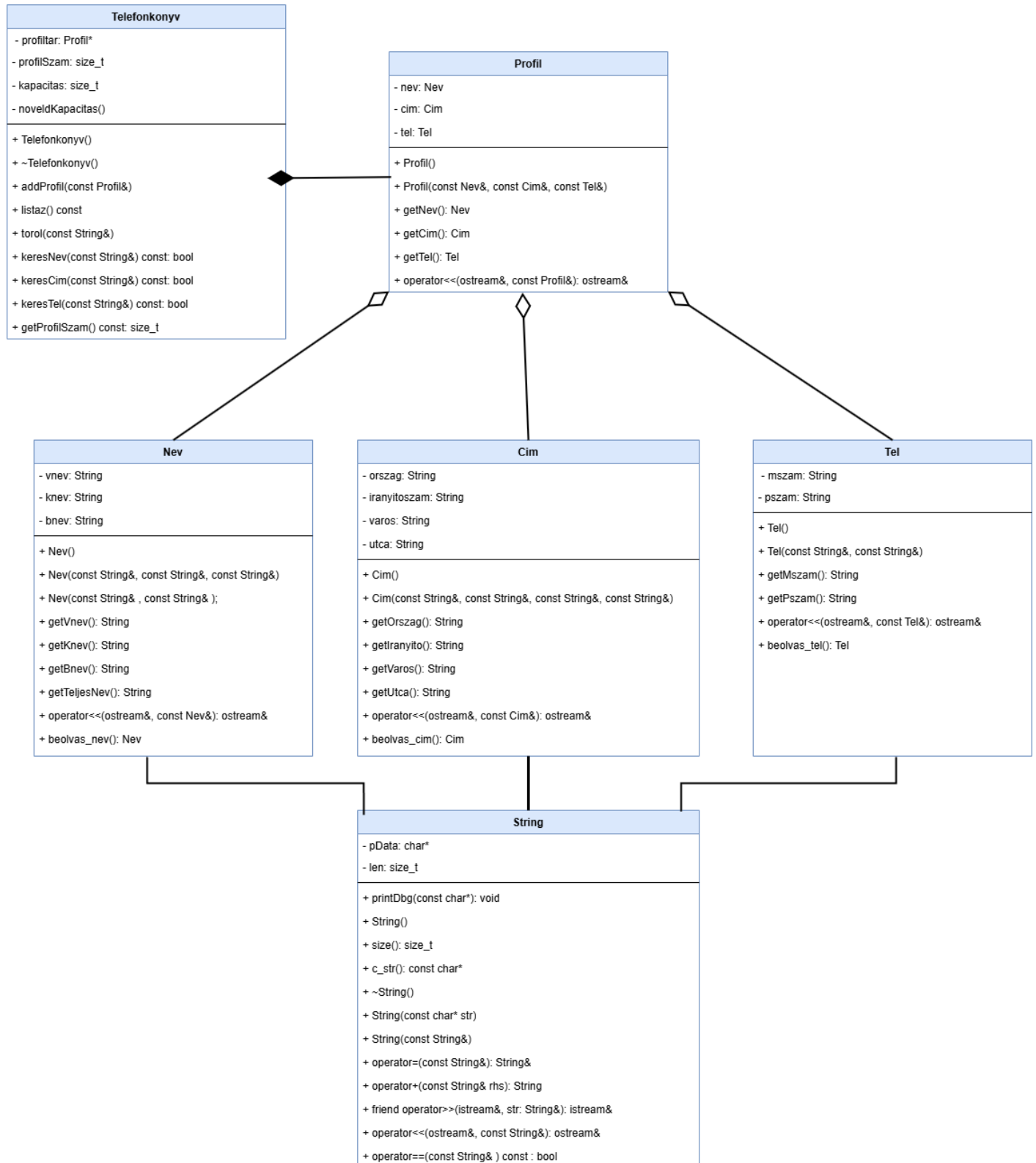
- **Konstruktorok** különböző paraméterezéssel
- **Getter metódusok** az adattagokhoz (**getNev()**, **getCim()**, **getTel()**)
- Operátor túlterhelés a kiíratáshoz (**operator<<**)

Az osztályok közötti kapcsolatok:

- A **Telefonkönyv** és **Profil** között **kompozíciós** kapcsolat van (kitöltött rombusz)
- A **Profil** és a **Nev**, **Cim**, **Tel** osztályok között **aggregációs** kapcsolat van (nem kitöltött rombusz)
- A **Nev**, **Cim**, **Tel** és **String** osztályok között **asszociációs** kapcsolat van (egyszerű vonal)
- Ez a struktúra lehetővé teszi a telefonkönyv rugalmas használatát, adatok hatékony kezelését és a bővíthetőséget is.

UML DIAGRAM:

Az osztálydiagramot a **draw.io** (diagrams.net) eszközzel készítettem, amely lehetővé tette számomra, hogy az UML jelölésekkel pontosan ábrázoljam az osztályok közötti kapcsolatokat és attribútumokat. Az elkészített diagramot exportáltam, és beillesztettem a dokumentumba, hogy bemutassam a telefonkönyv rendszer szerkezetét és működését.



3.2 Algoritmusok

A telefonkönyv optimális kapacitása, működés szempontjából „n” érték, amelyet a program figyelembe vesz, az n pontos értékét a tesztelés és az induló működés fogja meghatározni.

Profil hozzáadása

A telefonkönyvhez új profilt adunk hozzá, ha a kapacitás elérte a maximumot, növeljük a kapacitást, és másoljuk az adatokat.

Függvény addProfil(profil):

Ha profilSzam >= kapacitas akkor

Növeljük a kapacitást (noveldKapacitas())

Profilt adunk hozzá a profiltar tömbhöz az aktuális profilSzam indexre

Növeljük a profilSzamot

Kapacitás növelése

Ha a profilok száma meghaladja a tömb kapacitását, új tömböt hozunk létre, átmásoljuk az adatokat, majd töröljük a régi tömböt.

Függvény noveldKapacitas():

Növeljük a kapacitást 1-tel

Létrehozunk egy új Profil tömböt (ujTar) a növelt kapacitással

Átmásoljuk az adatokat a profiltárból ujTarba

Töröljük a régi profiltár tömböt

Beállítjuk profiltárat az ujTarra

Profil törlése név alapján

A keresett névvel rendelkező profilokat töröljük, majd az összes többi profilt eltoljuk, hogy a törölt helyet kitöltsük.

Függvény torol(vnev, knev):

Végig iterálunk a profiltar tömbön

Ha a profil neve megegyezik a keresett vnev és knev értékekkel:

Töröljük az aktuális profilt

Eltoljuk az összes következő profilt a tömbben

Csökkentsük a profilSzamot

Kiírunk egy üzenetet, hogy a törlés sikerült

Kilépés a ciklusból

Ha nem találtunk profilt:

Kiírunk egy üzenetet, hogy nem találtuk a profilt

Keresés név alapján

A keresett (név) alapján végig iterálunk a profilokon, és ha találunk egyezést, akkor kiírjuk a profil adatait.

Függvény keresNev(nev):

Beállítjuk a talalat változót hamisra

Végig iterálunk a profiltar tömbön

Ha a profil neve megegyezik a keresett névvel:

Kiírjuk a profilt

Beállítjuk a talalatot igazra

Ha nem találtunk találatot:

Kiírjuk az üzenetet, hogy nincs találat

Megjegyzés: ugyanez történik a cím és telefonszám alapján történő keresnél.

Név beolvasása

Lehetővé teszi a felhasználó számára, hogy megadja egy személy nevét, beleértve a vezetéknév, keresztnév és becenév mezőket, miközben biztosítjuk, hogy az adatok érvényesek.

Függvény beolvas_nev():

Kiírjuk a vezetéknévet

Beolvassuk a vezetéknévet

Ha vezetéknév érvényes:

Folytatjuk

Különben:

Kiírjuk hibát

Kiírjuk a keresztnévet

Beolvassuk a keresztnévet

Ha keresztnév érvényes:

Folytatjuk

Különben:

Kiírjuk hibát

Kiírjuk a becenevet

Beolvassuk becenevet

Visszatérünk a név objektummal

Megjegyzés: ugyanez történik a cím és telefonszám alapján történő beolvasásnál.

4. Megvalósítás

A telefonkönyv alkalmazás megvalósítása során a tervezési szakaszban meghatározott objektumorientált megközelítést követtem. A fejlesztés C++ nyelven történt, amely lehetővé tette a hatékony objektumorientált programozást és dinamikus memóriakezelést. A program nem használ STL tárolókat a specifikációnak megfelelően, ehelyett saját adatstruktúrákat valósít meg a dinamikus tároláshoz.

A feladat megoldása hat osztály és öt tesztprogram elkészítését igényelte. Az osztályokat a specifikációban részletezett módon valósítottam meg, ami lehetővé teszi a telefonkönyv alkalmazás megfelelő működését és a későbbi bővíthetőséget. A kód struktúrája moduláris felépítésű, minden osztály külön fejléc (.h) és implementációs (.cpp) fájlban található meg, amely elősegíti a kód karbantarthatóságát és átláthatóságát.

4.1 Osztályok bemutatása

Az alkalmazás hat fő osztályból épül fel, amelyek jól elkülönülő felelősségi körökkel rendelkeznek:

Osztály	Az összes adatszerkezet listája rövid leírásokkal:
String	A String osztály egy egyszerű implementációja a C-sztringek (nullával lezárt karakterláncok) kezelésére. Az osztály támogatja a dinamikus memória kezelést, másoló konstruktort, operátorokat és egyéb fontos műveleteket.
Nev	A Nev osztály tárolja és kezeli egy személy nevét (vezetéknév, keresztnév, becenév). A név különböző formáit (pl. teljes név) visszaadhatjuk, és az osztály tartalmaz konstruktorokat, hogy a nevet különböző módokon inicializáljuk.
Cim	A Cim osztály tárolja és kezeli egy cím adatait, beleértve az országot, irányítószámot, várost és utcát. Az osztály különböző konstruktorokat tartalmaz, és lehetővé teszi a címek különböző formájú beolvasását és kiírását.
Tel	A Tel osztály tárolja és kezeli egy személy telefonszámadatait, beleértve a munkahelyi és a privát telefonszámot. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve a telefonszámok kezelését, beolvasását és kiírását.
Profil	A Profil osztály tárolja és kezeli egy személy adatprofilját, beleértve a nevét, címét és telefonszámát. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve az adatok kezelését, beolvasását és kiírását.
Telefonkonyv	A Telefonkonyv osztály tárolja és kezeli a profilok listáját egy telefonkönyvben. Az osztály lehetővé teszi a profilok hozzáadását, törlését, keresését és listázását. Az adatok dinamikusán tárolódnak, és az osztály automatikusan kezeli a kapacitás növelését.

Az objektumtervben található rövid leírásokat érdemes bővebben, pontosabban és struktúráltabban kifejteni. Itt van egy javított változat, amely részletesebben bemutatja az osztály adattagjait és metódusait.

4.1.1 String osztályreferencia

A String osztály egy egyszerű implementációja a C-sztringek (nullával lezárt karakterláncok) kezelésére. Az osztály támogatja a dinamikus memória kezelést, másoló konstruktort, operátorokat és egyéb fontos műveleteket. A String osztályt az **5. laborfeladathoz** egészítettük ki és fejlesztettem tovább.

Adattagjai:

pData: pointer az adatra (char* típusú)

len: hossz, lezáró nulla nélkül (size_t típusú)

Publikus tagfüggvények

void printDbg (const char *txt="") const

Kiírunk egy Stringet (debug célokra) Ezt a tagfüggvényt elkészítettük, hogy használja a hibák felderítéséhez. Igény szerint módosítható

String()

Paraméter nélküli konstruktor

size_t size () const

Sztring hosszát adja vissza.

const char * c_str () const

C-sztringet ad vissza.

~String ()

Destruktor.

String (const char *str)

C-sztringből konstruktor.

String (const String &rhs)

Másoló konstruktor.

String & operator= (const String &rhs)

Másoló operátor.

String operator+ (const String &rhs) const

Összefűző operátor.

bool operator== (const String &other) const

Egyenlőség operátor.

Barát (friend)

`std::istream & operator>> (std::istream &is, String &str)`
Beolvasás.

Konstruktorok és destruktork

`String ()`

`~String ()` Felszabadítja az allokált memóriát.

`String (const char *str)`

Paraméter: `str` A bemeneti C-sztring, amely alapján az objektum inicializálásra kerül.

`String (const String &rhs)`

Paraméter: `rhs` A másolandó `String` objektum.

Tagfüggvények dokumentációja

`c_str() : const char * String::c_str () const`

C-sztringet ad vissza.

Visszatérési érték: pointer a tárolt, vagy azzal azonos tartalmú nullával lezárt sztring-re. `const char* c_str() const { return pData;}`

```
00034 { return pData ? pData : "";
```

`_operator+() : String String::operator+ (const String &rhs) const`

Összefűző operátor.

Paraméterek: `rhs` A másik `String` objektum, amellyel össze lesz fűzve.

Visszatérési érték: Az új összefűzött `String` objektum.

```
00057 {
00058     size_t ujLen = len + rhs.len;
00059     char* ujData = new char[ujLen + 1];
00060     strcpy(ujData, pData);
00061     strcat(ujData, rhs.pData);
00062     String eredmeny(ujData);
00063     delete[] ujData;
00064     return eredmeny;}
```

operator=() : **String** **String::operator=** (const **String** &**rhs**) const

Másoló operátor.

Paraméterek: **rhs** A másik **String** objektum.

Visszatérési érték: A hivatkozott **String** objektum.

```
00037         {
00038     if (this != &rhs) {
00039         delete[] pData;
00040     if (rhs.pData) {
00041         len = rhs.len;
00042         pData = new char[len + 1];
00043         strcpy(pData, rhs.pData);
00044     } else {
00045         len = 0;
00046         pData = nullptr;
00047     }
00048 }
00049 }
00050 return *this;
00051 }
```

operator==() : **String** **String::operator=** (const **String** &**other**) const

Egyenlőség operátor.

Paraméterek: **other** A másik **String** objektum, amivel az egyenlőséget vizsgáljuk.

Visszatérési érték: Igaz, ha a két **String** objektum egyenlő, egyébként hamis.

```
00075         {
00076     if (len != other.len) return false;
00077     return strcmp(pData, other.pData) == 0;
00078 }
```

size() : **size_t** **String::size** () const

Sztring hosszát adja vissza.

Visszatérési érték: sztring tényleges hossza (lezáró nulla nélkül).

```
00029 { return len; }
```

printDbg() : void String::printDbg (const char * **txt** = "") const

Kiírunk egy Stringet (debug célokra) Ezt a tagfüggvényt elkészítettük, hogy használja a hibák felderítéséhez. Igény szerint módosíthat

Paraméterek: txt- nullával lezárt szövegre mutató pointer. Ezt a szöveget írjuk ki a debug információ előtt.

```
00019      {
00020      std::cout << txt << "[" << len << "], "
00021      << (pData ? pData : "(NULL)") << '|' << std::endl;
00022  }
```

Barát és kapcsolódó függvények dokumentációja

operator>> : std::istream & operator>> (std::istream &**is**, String & **str**)

Beolvasás.

Paraméterek:

is A bemeneti stream, amelyből beolvassuk az adatokat.

str Az a **String** objektum, amelybe beírjuk a beolvasott adatokat.

```
00081      {
00082      delete[] str.pData;
00083      str.pData = nullptr;
00084      str.len = 0;
00085
00086      char* temp = nullptr;
00087      char ch;
00088      size_t index = 0;
00089
00090      while (is.get(ch) && ch != '\n') {
00091          char* newTemp = new char[index + 2];
00092          if (temp) {
00093              memcpy(newTemp, temp, index);
00094              delete[] temp;
00095          }
00096          newTemp[index] = ch;
00097          newTemp[index + 1] = '\0';
00098          temp = newTemp;
00099          index++;
00100      }
00101      str.len = index;
00102      str.pData = temp;
00103      return is;
00104 }
```


4.1.2 Nev osztályreferencia

A Nev osztály tárolja és kezeli egy személy nevét (vezetéknév, keresztnév, becenév). A név különböző formáit (pl. teljes név) visszaadhatjuk, és az osztály tartalmaz konstruktorokat, hogy a nevet különböző módokon inicializáljuk

Adattagjai:

vnev: Vezetéknév (String típusú)

knev: Keresztnév (String típusú)

bnev: Becenév (String típusú)

Publikus tagfüggvények

Nev ()

Alapértelmezett konstruktor.

Nev (const String &vnev, const String &knev, const String &bnev)

Konstruktor három név paraméterrel (vezeték, keresztnév, becenév).

Nev (const String &vnev, const String &knev)

Konstruktor két név paraméterrel (vezeték, keresztnév).

String getVnev () const

Visszaadja a vezetéknév értékét.

String getKnev () const

Visszaadja a keresztnév értékét.

String getBnev () const

Visszaadja a becenév értékét.

String getTeljesNev () const

Visszaadja a teljes nevet (vezeték + keresztnév).

Nev beolvas_nev ()

Név beolvasása a felhasználótól.

Konstruktorok és dokumentációja

Nev ()

Alapértelmezett konstruktor.

Nev (const String &vnev, const String &knev, const String &bnev)

Konstruktor három név paraméterrel (vezeték, keresztnév, becenév).

Paraméterek:

vnev: A vezetéknév

knev: A keresztnév

bnev: A becenév

Nev (const String &vnev, const String &knev)

Konstruktor két név paraméterrel (vezeték, keresztnév).

Paraméterek:

vnev: A vezetéknév

knev: A keresztnév

beolvas_nev() : Nev Nev::beolvas_nev()

Név beolvasása a felhasználótól.

Visszatérési érték: A beolvasott **Nev** objektum.

-Vezetéknév beolvasása

Érvényes keresztnév, ha legalább egy karakter és csak betűket tartalmaz

-Keresztnév beolvasása

Érvényes keresztnév, ha legalább egy karakter és csak betűket tartalmaz

-Becenév beolvasása (opcionális, lehet üres)

```

00021     {
00022     String v, k, b;
00023     std::cin.clear();
00025     while (true) {
00026         std::cout << "Vezeteknev: ";
00027         std::cin >> v;
00028         if (v.size() > 0 && std::all_of(v.c_str(), v.c_str() + v.size(), ::isalpha)) {
00029             break;
00030         } else {
00031             std::cout << "Hiba: A vezeteknevnek legalabb 1 betubol kell allnia, es csak betu lehet!"
<< std::endl;
00032         }
00033     }
00034
00036     while (true) {
00037         std::cout << "Keresztnev: ";
00038         std::cin >> k;
00039         if (k.size() > 0 && std::all_of(k.c_str(), k.c_str() + k.size(), ::isalpha)) {
00040             break;
00041         } else {
00042             std::cout << "Hiba: A keresztnevnek legalabb 1 betubol kell allnia, es csak betu lehet!"
<< std::endl;
00043         }
00044     }
00045
00047     std::cout << "Becenev: ";
00048     std::cin >> b;
00049
00050     return Nev(v, k, b);
00051 }

```

getVnev() : String Nev::getVnev () const

Visszaadja a vezetéknév értékét.
Visszatérési érték: A vezetéknév.

```
00014 { return vnev; }
```

getKnev() : String Nev::getKnev () const

Visszaadja a keresztnév értékét.
Visszatérési érték: A keresztnév.

```
00015 { return knev; }
```

getBnev() : String Nev::getBnev () const

Visszaadja a becenév értékét.
Visszatérési érték: A becenév.

```
00016 { return bnev; }
```

getTeljesNev() : String Nev::getTeljesNev () const

Visszaadja a teljes nevet (vezeték + keresztnév).
Visszatérési érték: A teljes név.

```
00017 { return vnev + " " + knev; }
```

4.1.3 Cim osztályreferencia

A Cim osztály tárolja és kezeli egy cím adatait, beleértve az országot, irányítószámot, várost és utcát. Az osztály különböző konstruktorokat tartalmaz, és lehetővé teszi a címek különböző formájú beolvasását és kiírását.

Adattagjai:

ország: Ország megnevezése (String típusú)

iranyitoszam: Irányítószám (String típusú)

varos: Város neve (String típusú)

utca: Utca neve (String típusú)

Publikus tagfüggvények

Cim ()

Alapértelmezett konstruktor.

Cim (const String &ország, const String &iranyitoszam, const String &varos, const String &utca)

Konstruktor a cím négy paraméterrel (ország, irányítószám, város, utca).

String getOrszag () const

Visszaadja az országot.

String getIrandito () const

Visszaadja az irányítószámot.

String getVaros() const

Visszaadja a várost.

String getUtca () const

Visszaadja az utcát.

Cim beolvas_cim ()

Beolvassa a címet a felhasználtól.

Konstruktorok és dokumentációja

Cim ()

Alapértelmezett konstruktor.

Cim (const String &ország, const String &iranyitoszam, const String &varos, const String &utca)

Konstruktor a cím négy paraméterrel (ország, irányítószám, város, utca).

Paraméterek:

ország: Az ország neve

iranyitoszam: Az irányítószám

varos: A város neve

utca: Az utca neve

beolvas_cim() : Cim Cim::beolvas_cim()

Cím beolvasása a felhasználótól.

Visszatérési érték: A beolvasott **Cim** objektum.

-Ország beolvasása
-Írányítószám beolvasása
-Város beolvasása
-Utca beolvasása

```

00022     {
00023     String orsz, irsz, var, utc;
00024
00026     while (true) {
00027         std::cout << "Orszag: ";
00028         std::cin >> orsz;
00029         if (orsz.size() > 0 && std::all_of(orsz.c_str(), orsz.c_str() + orsz.size(), ::isalpha)) {
00030             break;
00031         } else {
00032             std::cout << "Hiba: Az orszag neve nem lehet ures, es nem tartalmazhat szamokat,
szokozokat." << std::endl;
00033         }
00034     }
00035
00037     while (true) {
00038         std::cout << "Irányitoszam (4 karakter, csak szamok): ";
00039         std::cin >> irsz;
00040         if (irsz.size() == 4 && std::all_of(irsz.c_str(), irsz.c_str() + 4, ::isdigit)) {
00041             break;
00042         } else {
00043             std::cout << "Hiba: Az irányitoszamnak 4 számjegyből kell állnia." << std::endl;
00044         }
00045     }
00046
00048     while (true) {
00049         std::cout << "Varos: ";
00050         std::cin >> var;
00051         if (var.size() > 0 && std::all_of(var.c_str(), var.c_str() + var.size(), ::isalpha)) {
00052             break;
00053         } else {
00054             std::cout << "Hiba: A varos neve nem lehet ures, es nem tartalmazhat szamokat, szokozokat." <<
std::endl;
00055         }
00056     }
00057
00059     while (true) {
00060         std::cout << "Utca: ";
00061         std::cin >> utc;
00062         if (utc.size() > 0) {
00063             break;
00064         } else {
00065             std::cout << "Hiba: Az utca neve nem lehet ures" << std::endl;
00066         }
00067     }
00068
00069     return Cim(orsz, irsz, var, utc);
00070 }

```

getIrányito() : String Cim::getIrányito () const

Visszaadja az irányítószámot.
Visszatérési érték: Az irányítószám.

```
00017 { return irányitoszam; }
```

getOrszag() : String Cim::getOrszag () const

Visszaadja az országot.
Visszatérési érték: Az ország neve.

```
00016 { return orszag; }
```

getUtca() : String Cim::getUtca () const

Visszaadja az utcát.
Visszatérési érték: Az utca neve.

```
00019 { return utca; }
```

getVaros() : String Cim::getVaros () const

Visszaadja a várost.
Visszatérési érték: A város neve.

```
00018 { return varos; }
```

4.1.4 Tel osztályreferencia

A Tel osztály tárolja és kezeli egy személy telefonszámadatait, beleértve a munkahelyi és a privát telefonszámot. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve a telefonszámok kezelését, beolvasását és kiírását.

Adattagjai:

mszam: Munkahelyi szám (String típusú)

pszam: Privát szám (String típusú)

Publikus tagfüggvények

Tel ()

Alapértelmezett konstruktor.

Tel (const String &mszam, const String &pszam)

Konstruktor két telefonszám paraméterrel (munkahelyi, privát).

String getMszam () const

Visszaadja a munkahelyi telefonszámot.

String getPszam () const

Visszaadja a privát telefonszámot.

Tel beolvas_tel ()

Beolvassa a telefonszámokat a felhasználtól.

Konstruktorok és dokumentációja

Tel ()

Alapértelmezett konstruktor.

Tel (const String &mszam, const String &pszam)

Konstruktor két telefonszám paraméterrel (munkahelyi, privát).

Paraméterek:

mszam: A munkahelyi telefonszám

pszam: A privát telefonszám

beolvas_tel() : Tel Tel::beolvas_tel ()

Telefonszám beolvasása a felhasználótól.

Visszatérési érték: A beolvasott **Tel** objektum.

- Munkahelyi szám beolvasása

Ellenőrizzük, hogy a bemenet 11 karakter hosszú és csak számokat tartalmaz

Ha érvényes, kilépünk a ciklusból

-Privát szám beolvasása

Ellenőrizzük, hogy a bemenet 11 karakter hosszú és csak számokat tartalmaz

Ha érvényes, kilépünk a ciklusból

```

00022     {
00023     String msz, psz;
00024
00026     while (true) {
00027         std::cout << "Munkahelyi szam (11 karakter, csak szamok): ";
00028         std::cin >> msz;
00029
00031         if (msz.size() == 11 && std::all_of(msz.c_str(), msz.c_str() + 11, ::isdigit)) {
00032             break;
00033         } else {
00034             std::cout << "Hiba: A munkahelyi szamnak 11 szamjegyből kell állnia." << std::endl;
00035         }
00036     }
00037
00039     while (true) {
00040         std::cout << "Privat szam (11 karakter, csak szamok): ";
00041         std::cin >> psz;
00042
00044         if (psz.size() == 11 && std::all_of(psz.c_str(), psz.c_str() + 11, ::isdigit)) {
00045             break;
00046         } else {
00047             std::cout << "Hiba: A privat szamnak 11 szamjegyből kell állnia." << std::endl;
00048         }
00049     }
00050
00051     return Tel(msz, psz);
00052 }
    
```



```
getMszam() :: String Tel::getMszam ( ) const
```

Visszaadja a munkahelyi telefonszámot.

Visszatérési érték: A munkahelyi telefonszám.

```
00017 { return mszam; }
```

```
getPszam() : String Tel::getPszam ( ) const
```

Visszaadja a privát telefonszámot.

Visszatérési érték: A privát telefonszám.

```
00018 { return pszam; }
```

4.1.5 Profil osztályreferencia

A Profil osztály tárolja és kezeli egy személy adatprofilját, beleértve a nevét, címét és telefonszámát. Az osztály különböző konstruktorokat tartalmaz, lehetővé téve az adatok kezelését, beolvasását és kiírását.

Adattagjai:

nev: Nev típusú objektum

cim: Cim típusú objektum

tel: Tel típusú objektum

Publikus tagfüggvények

Profil ()

Alapértelmezett konstruktor.

Profil (const Nev &nev, const Cim &cím, const Tel &tel)

Konstruktor két telefonszám paraméterrel (munkahelyi, privát).

Nev getNev () const

Visszaadja a személy nevét.

Cim getCim () const

Visszaadja a személy címét.

Tel getTel () const

Visszaadja a személy telefonszámát.

Konstruktorok

Profil ()

Alapértelmezett konstruktor.

Profil (const Nev &nev, const Cim &cím, const Tel &tel)

Konstruktor két telefonszám paraméterrel (munkahelyi, privát).

Paraméterek:

nev: A személy neve

cim: A személy címe

tel: A személy telefonszáma

getNev() : Nev Profil::getNev () const

Visszaadja a személy nevét.

Visszatérési érték: A személy neve (**Nev** típusú objektum).

```
00014 { return nev; }
```

getTel() : Tel Profil::getTel () const

Visszaadja a személy telefonszámát.

Visszatérési érték: A személy telefonszáma (**Tel** típusú objektum).

```
00016 { return tel; }
```

getCim() : Cim Profil::getCim () const

Visszaadja a személy címét.

Visszatérési érték: A személy címe (**Cim** típusú objektum).

```
00015 { return cim; }
```

4.1.6 Telefonkönyv osztályreferencia

A Telefonkönyv osztály tárolja és kezeli a profilok listáját egy telefonkönyvben. Az osztály lehetővé teszi a profilok hozzáadását, törlését, keresését és listázását. Az adatok dinamikusan tárolódnak, és az osztály automatikusan kezeli a kapacitás növelését.

Adattagjai:

profiltar: Profil típusú objektumokat tároló pointer
profilSzam: A jelenleg tárolt profilok száma (size_t típusú)
kapacitas: A telefonkönyv maximális kapacitása (size_t típusú)

Publikus tagfüggvények

Telefonkönyv ()
Alapértelmezett konstruktor, amely inicializálja a telefonkönyvet alapértelmezett értékekkel.

~Telefonkönyv ()
Destruktor, amely felszabadítja a dinamikusan allokalált memóriát.

void addProfil (const Profil &profil)
Adatok felvétele.

void listaz () const
A telefonkönyvben tárolt profilok listázása a kimenetre.

void torol (const String &nev, const String &knev)
Profil törlése a telefonkönyvből a név alapján.

bool keresNev (const String &nev) const
Keresés név alapján.

bool keresCim (const String &cim) const
Keresés cím alapján.

bool keresTel (const String &tel) const
Keresés telefonszám alapján.

size_t getProfilSzam () const
A telefonkönyvben tárolt profilok számának lekérdezése.

Konstruktor és destruktor / dokumentációja

Telefonkönyv ()
Alapértelmezett konstruktor.

~Telefonkönyv ()
Destruktor, amely felszabadítja a dinamikusan allokalált memóriát.

addprofil() : void Telefonkonyv::addProfil (const Profil& profil)

Adatok felvétele.
 Profil hozzáadása
 Profilok számának növelése

```
00028         {
00029     if (profilSzam >= kapacitas) {
00030         noveldKapacitas();
00031     }
00032     profiltar[profilSzam] = profil;
00033     profilSzam++;
00034 }
```

getProfilSzam() : size_t Telefonkonyv::getProfilSzam() const

A telefonkönyvben tárolt profilok számának lekérdezése.
 Profilok számának lekérdezése.

```
00115         {
00116     return profilSzam;
00117 }
```

keresCim() : bool Telefonkonyv::keresCim (const String& cim) const

Keresés cím alapján.
 Paraméterek: **cim** A keresett cím.
 Visszatérési érték: Igaz, ha a cím megtalálható, egyébként hamis.

```
00081         {
00082     bool talalat = false;
00083     for (size_t i = 0; i < profilSzam; i++) {
00084         if (profiltar[i].getCim().getUtca() == cim ||
00085             profiltar[i].getCim().getVaros() == cim ||
00086             profiltar[i].getCim().getOrszag() == cim ||
00087             profiltar[i].getCim().getIrandito() == cim) {
00088             std::cout << profiltar[i] << std::endl;
00089             talalat = true;
00090         }
00091     }
00092     if (!talalat) {
00093         std::cout << "Nincs talalat a(z) " << cim << " cimre." << std::endl;
00094     }
00095     return talalat;
00096 }
```

keresNev() : bool Telefonkonyv::keresNev (const String& nev) const

Keresés név alapján. (Egyszerű keresés)

Paraméterek: **nev** A keresett név.

Visszatérési érték: Igaz, ha a név megtalálható, egyébként hamis.

```
00063         {
00064     bool talalat = false;
00065     for (size_t i = 0; i < profilSzam; i++) {
00066         if (profiltar[i].getNev().getVnev() == nev ||
00067             profiltar[i].getNev().getKnev() == nev ||
00068             profiltar[i].getNev().getTeljesNev() == nev ||
00069             profiltar[i].getNev().getBnev() == nev) {
00070             std::cout << profiltar[i] << std::endl;
00071             talalat = true;
00072         }
00073     }
00074     if (!talalat) {
00075         std::cout << "Nincs talalat a(z) " << nev << " nevre." << std::endl;
00076     }
00077     return talalat;
00078 }
```

keresTel() : bool Telefonkonyv::keresTel (const String& tel) const

Keresés telefonszám alapján.

Paraméterek: **tel** A keresett telefonszám.

Visszatérési érték: Igaz, ha a tel megtalálható, egyébként hamis.

```
00099         {
00100     bool talalat = false;
00101     for (size_t i = 0; i < profilSzam; i++) {
00102         if (profiltar[i].getTel().getMszam() == tel ||
00103             profiltar[i].getTel().getPszam() == tel) {
00104             std::cout << profiltar[i] << std::endl;
00105             talalat = true;
00106         }
00107     }
00108     if (!talalat) {
00109         std::cout << "Nincs talalat a(z) " << tel << " telefonszamra." << std::endl;
00110     }
00111     return talalat;
00112 }
```

listaz() : Telefonkonyv::listaz () const

A telefonkönyvben tárolt profilok listázása a kimenetre.
Listázás.

```
00037         {
00038     for (size_t i = 0; i < profilSzam; i++) {
00039         std::cout << profiltar[i] << std::endl;
00040     }
00041 }
```

torol() : void Telefonkonyv::torol (const String &vnev, const String &knev)

Profil törlése a telefonkönyvből a név alapján.
Adatok törlése.

Paraméterek:

vnev A keresett személy vezetéknév része.

knev A keresett személy keresztnév része.

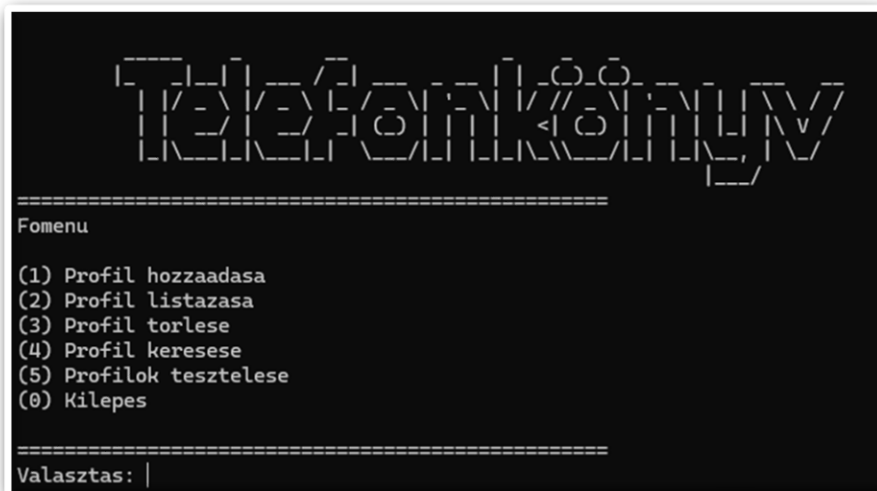
```
00044         {
00045     bool talalt = false;
00046     for (size_t i = 0; i < profilSzam; i++) {
00047         if (profiltar[i].getNev().getVnev() == vnev && profiltar[i].getNev().getKnev() == knev) {
00049             for (size_t j = i; j < profilSzam - 1; j++) {
00050                 profiltar[j] = profiltar[j + 1];
00051             }
00052             profilSzam--;
00053             std::cout << "Profil torolve: " << vnev << " " << knev << std::endl;
00054             talalt = true;
00055         }
00056     }
00057     if (!talalt){
00058         std::cout << "Profil nem talalhato: " << vnev << " " << knev << std::endl;
00059     }
00060 }
```

4.2 Forráskód és dokumentáció

A forráskód Doxygen dokumentációs rendszerrel készült, amely lehetővé teszi a kód struktúrájának és funkcióinak áttekintését. A dokumentáció HTML és PDF formátumban is elérhető. A megvalósítás során nagy hangsúlyt fektettem az olvasható, könnyen karbantartható kódra, valamint a megfelelő hibakezelésre és a felhasználók számára értelmezhető visszajelzésekre.

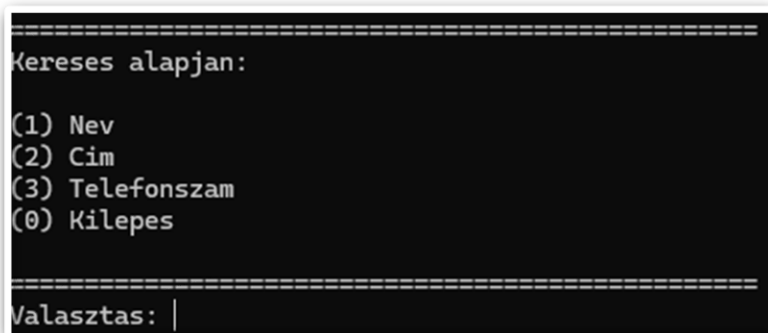
4.3 Menüvezérelt felhasználói felület

A telefonkönyv alkalmazás felhasználói felületét menüvezérelt rendszerként valósítottam meg, amely lehetővé teszi a program intuitív, könnyű használatát. A menüben használt „profil” elnevezés egy valós személy telefonkönyvi adatait, vezetéknév, keresztnév, becenév, cím (ország, irányítószám, város, utca), valamint a telefonszámokat (munkahelyi és privát) tartalmazza.



A menüpontok részletesebben:

1. **Profil hozzáadása:** A felhasználó itt adhat új profilt a telefonkönyvhöz. Interaktív módon bekéri a név adatokat (vezetéknév, keresztnév, becenév), a cím részleteit (ország, irányítószám, város, utca), valamint a telefonszámokat (munkahelyi és privát).
2. **Profil listázása:** Az összes tárolt profil áttekinthető listázása a konzolra.
3. **Profil törlése:** Lehetőséget biztosít a felhasználónak, hogy név alapján töröljön profilt. A vezetéknév és keresztnév megadásával azonosítja a törölni kívánt profilt.
4. **Profil keresése:** Ez a funkció almenüt nyit, ahol a felhasználó választhat a különböző keresési módok között:



5. **Profilok tesztelése:** A tesztfunkciók futtatása, amelyek ellenőrzik az alkalmazás¹különböző aspektusainak helyes működését.

4.3 Hibakezelés

A Telefonkönyv alkalmazásban komplex hibakezelési rendszert implementáltam, amely biztosítja a program stabil és felhasználóbarát működését. A hibakezelési mechanizmusok kiterjednek az adatbevitel validálására, a memóriakezelésre, a felhasználói interakciókra és a program tesztelésére is.

Adatbeviteli hibák kezelése:

Névadatok ellenőrzése (Nev osztály):

Vezetéknév és keresztnév validálása:

- **Hibaüzenet:** A vezeteknevnek legalabb 1 betubol kell allnia, es csak betu lehet!
- **Hibaüzenet:** A keresztnevnek legalabb 1 betubol kell allnia, es csak betu lehet!

Az alkalmazás ellenőrzi, hogy a név csak betűket tartalmazzon és ne legyen üres.

Címadatok ellenőrzése (Cim osztály)

Ország és város ellenőrzése:

- **Hibaüzenet:** Az orszag neve nem lehet ures, es nem tartalmazhat szamokat, szokozokat.
- **Hibaüzenet:** A varos neve nem lehet ures, es nem tartalmazhat szamokat, szokozokat.

Több tagból álló esetben a tagok között a számítástechnikában alkalmazott „_” jelet kell használni. pl: New_York

Irányítószám formátum validálása:

- **Hibaüzenet:** Az irányitoszamnak 4 számjegyből kell állnia.

Utcanév ellenőrzése:

- **Hibaüzenet:** Az utca neve nem lehet ures

Telefonszámok validálása (Tel osztály)

Telefonszám formátum ellenőrzése:

- **Hibaüzenet:** A munkahelyi számnak 11 számjegyből kell állnia.
- **Hibaüzenet:** A privat számnak 11 számjegyből kell állnia.

¹ Megjegyzés: A program indításakor 3 db minta-profil automatikusan feltöltésre kerül a telefonkönyvbe

Az alkalmazás ellenőrzi, hogy a telefonszám a magyarországi szabványos 11 számjegyből áll.

Telefonkönyv műveletekhez kapcsolódó hibakezelés

Törlési műveletek hibakezelése

Nem létező profil törlésénél:

- **Profil nem található:** [vezetéknév] [keresztnev]

Sikeres törlés visszaigazolása:

- **Profil törölve:** [vezetéknév] [keresztnev]

Keresési műveletek hibakezelése

Sikertelen név szerinti keresés:

- **Nincs talalat a(z) [név] nevre.**

Sikertelen cím szerinti keresés:

- **Nincs talalat a(z) [cím] címre.**

Sikertelen telefonszám szerinti keresés:

- **Nincs talalat a(z) [telefonszám] telefonszámra.**

Felhasználói interfész hibakezelése

Menürendszer validálása: A main.cpp-ben látható menürendszer ellenőrzi a felhasználói bemeneteket, és érvénytelen választás esetén hibaiüzenetet jelenít meg:

- **Hibaiüzenet: Ervénytelen valasz, probald ujra.**

Tesztelési rendszer

- A program beépített tesztrendszert tartalmaz a gtest_lite.h segítségével, amely lehetővé teszi a funkciók automatikus tesztelését és a hibák korai felderítését.
- A tesztek() funkció (amelyet az 5-ös menüpont aktivál) lehetőséget biztosít az implementált osztályok és funkciók tesztelésére, biztosítva, hogy a hibakezelés minden szinten megfelelően működik.

A program nem használ C++ kivételkezelést (try, catch kulcsszavakat), mivel a cél egy egyszerű és könnyen átlátható hibakezelési logika megvalósítása volt.

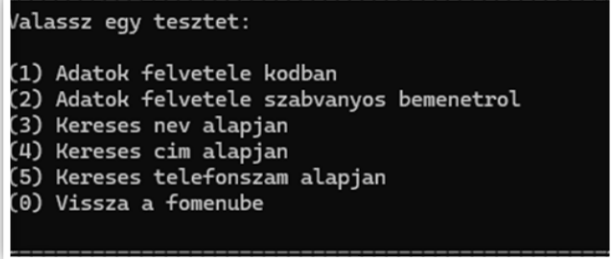
5. Tesztelés

A program helyes működésének ellenőrzésére több automatikus tesztet készítettem a **gtest_lite** keretrendszer használatával. A teszteléshez külön .h és .cpp fájlok is készültek, a tesztek célja, hogy automatikusan validálják a különböző modulok (például a profilkezelés, keresés, törlés stb.) helyes működését.

A tesztek lefedik az alábbi főbb eseteket:

- profilok hozzáadása és törlése,
- név, cím, telefonszám alapján történő keresés,
- szabványos bemenetről történő adatfelvétel.

A tesztelés során a program a várt eredményeket hasonlítja össze a tényleges működés eredményeivel. A tesztek minden lépése során szöveges visszajelzést kapunk, valamint a **EXPECT_EQ** és **EXPECT_TRUE/FALSE** makrók segítségével ellenőrizzük az elvárt és tényleges kimeneteket. A **clearScreen()** és **waitForEnter()** segédfüggvények biztosítják a felhasználóbarát, átlátható futást.



```
Valassz egy tesztet:  
(1) Adatok felvetele kodban  
(2) Adatok felvetele szabvanyos bemenetrol  
(3) Kereses nev alapjan  
(4) Kereses cim alapjan  
(5) Kereses telefonszam alapjan  
(0) Vissza a fomenube
```

Függvények

void kiir (int hossz, char c)

Sor kiírása a képernyőre.

void clearScreen ()

Képernyő törlése a megfelelő operációs rendszeren.

void waitForEnter ()

Várakozás az ENTER gomb lenyomására.

void teszt_1 ()

Teszt 1: Adatok felvitele kódban.

void teszt_2 ()

Teszt 2: Adatok felvétele szabványos bemenetről (felhasználói input)

void teszt_3 ()

Teszt 3: Keresés név alapján.

void teszt_4 ()

Teszt 4: Keresés cím alapján.

void teszt_5 ()

Teszt 5: Keresés telefonszám alapján

void **clearScreen** ()

Képernyő törlése a megfelelő operációs rendszeren.

Ez a függvény a képernyőt törli a rendszertől függően. Ha Windows rendszeren fut a program, akkor a cls parancsot használja a képernyő törlésére. Ha Linux vagy Mac rendszer alatt fut, akkor a clear parancsot hívja meg.

Linux/Mac rendszeren

```
00019 {
00020 #ifdef _WIN32
00021     system("cls");
00022 #else
00023     system("clear");
00024 #endif
00025 }
```

void **kiir** (int hossz, char c)

Sor kiírása a képernyőre.

Ez a függvény a megadott karakterből és hosszúsággal kiír egy sort a képernyőre. A függvény addig ismétli a karakter kiírását, amíg el nem éri a kívánt hosszúságot.

Paraméterek:

hossz: A kiírandó sor hossza

c: A kiírandó karakter

```
00012 {
00013     for (int i = 0; i < hossz; i++) {A
00014         std::cout << c;
00015     }
00016     std::cout << std::endl;
00017 }
```

```
void waitForEnter ( )
```

Várakozás az ENTER gomb lenyomására.

Ez a függvény üzenetet ír ki, amely felkéri a felhasználót, hogy nyomja meg az ENTER gombot a folytatáshoz. Miután a felhasználó megnyomja az ENTER-t, a program folytatódik.

```
00027      {
00028  std::cout << "Nyomj meg egy ENTER-t a folytatashoz..." << std::endl;
00029
00030  fflush(stdin);
00031  getchar();
00032
00033 }
```

5.1 teszt_1 ()

Teszt 1: Adatok felvitele kódban.

Ez a teszt a telefonkönyv adatainak programkódból történő felvételét és törlését vizsgálja. Létrehoztam néhány profil adatot, amelyeket hozzáadunk a „telefonkönyvhöz”, majd ellenőrizzük a profilok számát. Ezután törlünk egy profilt, és ismét ellenőrizzük a profilok számát.

```
---> Telefonkonyv.AdatokFelvetele
=====
1. teszt - Adatok felvetele kodban

Profilok listazasa:
Nev: Kiss Kata (KisKata)
Cim: 4400 Nyiregyhaza, Kiss Pal utca 6, Magyarország
Munkahelyi es Privat: 123456789 / 987654321

Nev: Nagy Noemi (Noe)
Cim: 4025 Debrecen, Kossuth Lajos utca 2, Magyarország
Munkahelyi es Privat: 891238485 / 535664646

Nev: Kolos Vera (Vera)
Cim: 2234 Mogyorod, Tep utca 4, Magyarország
Munkahelyi es Privat: 06295760940 / 06448599209
=====
```

```
=====
Profil torlese (Kiss Kata):
Profil torolve: Kiss Kata

Profilok listazasa:
Nev: Nagy Noemi (Noe)
Cim: 4025 Debrecen, Kossuth Lajos utca 2, Magyarország
Munkahelyi es Privat: 891238485 / 535664646

Nev: Kolos Vera (Vera)
Cim: 2234 Mogyorod, Tep utca 4, Magyarország
Munkahelyi es Privat: 06295760940 / 06448599209
=====
Kilepes az elso tesztbol! (ENTER)
Nyomj meg egy ENTER-t a folytatashoz...
```

```
SIKERES    Telefonkonyv.AdatokFelvetele <---
```

5.2 teszt_2 ()

Teszt 2: Adatok felvétele szabványos bemenetről (felhasználói input)

Ez a teszt azt vizsgálja, hogyan lehet adatokat felvenni a telefonkönyvbe a felhasználói bemenet segítségével. A felhasználó meghatározza, hány profilt szeretne hozzáadni, majd a program bekéri a szükséges adatokat (név, cím, telefon). A teszt végén ellenőrizzük, hogy a profilok száma megfelelően megnövekedett.

```
---> Telefonkonyv.Adatok_Felvetele_Szabvanyos_Bemenetrol
=====

2. teszt - Adatok felvetele szabvanyos bemenetrol
Hany profilt szeretnel felvenni? 1

Profil 1. felvetele:
Vezeteknev: Teszt
Keresztnev: Elek
Becenév: Teszti
Ország: Magyarország
Iranyitoszam (4 karakter, csak számok): 1234
Varos: Gyor
Utca: Janos 8
Munkahelyi szám (11 karakter, csak számok): 12345678910
Privat szám (11 karakter, csak számok): 12345678910

Profilok listazasa:
Nev: Teszt Elek (Teszti)
Cim: 1234 Gyor, Janos 8, Magyarország
Munkahelyi es Privat: 12345678910 / 12345678910
=====
```

SIKERES Telefonkonyv.Adatok_Felvetele_Szabvanyos_Bemenetrol <---

5.3 teszt_3 ()

Teszt 3: Keresés név alapján.

Ebben a tesztben a telefonkönyvben keresünk egy adott név alapján. A teszt biztosítja, hogy a keresési funkció megfelelően működjön, és a keresett profilt visszaadja. A teszt során több profilt adunk hozzá, majd a keresés eredményét ellenőrizzük.

```
---> Telefonkonyv.Kereses_Nev_Alapjan
=====

3. teszt - Kereses (nev)
Kereses teljes nev alapjan (Kiss Kata):
Nev: Kiss Kata (KissKata)
Cim: 4400 Nyiregyhaza, Kiss Pal utca 6, Magyarország
Munkahelyi es Privat: 123456789 / 987654321

Kereses vezeteknev alapjan (Kiss):
Nev: Kiss Kata (KissKata)
Cim: 4400 Nyiregyhaza, Kiss Pal utca 6, Magyarország
Munkahelyi es Privat: 123456789 / 987654321

Kereses keresztnev alapjan (Noemi):
Nev: Nagy Noemi (Noe)
Cim: 4025 Debrecen, Kossuth Lajos utca 2, Nemetszag
Munkahelyi es Privat: 891238485 / 535664646

Kereses becenev alapjan (Vera):
Nev: Kolos Vera (Vera)
Cim: 2234 Mogyorod, Tep utca 4, Magyarország
Munkahelyi es Privat: 06295760940 / 06448599209

Kereses nem letezo nev alapjan (Nem letezo Nev):
Nincs talalat a(z) Anna nevre.
```

```
Profil torlese (Kiss Kata):
Profil torolve: Kiss Kata
Kereses nev alapjan (Kiss) torles utan:
Nincs talalat a(z) Kiss nevre.
=====
```

SIKERES Telefonkonyv.Kereses_Nev_Alapjan <---

5.4 teszt_4 ()

Teszt 4: Keresés cím alapján.

Ez a teszt a telefonkönyvben való keresést vizsgálja egy adott cím alapján. A teszt során különböző címekkel rendelkező profilokat adunk hozzá a telefonkönyvhöz, majd megpróbálunk keresni egy adott címet. A keresett profil(ok) helyességét ellenőrizzük.

```
----> Telefonkonyv.Kereses_Cim_Alapjan
=====

4. teszt - Kereses (cim)
Kereses utca alapjan (Kossuth Lajos utca 2):
Nev: Nagy Noemi (Noe)
Cim: 4025 Debrecen, Kossuth Lajos utca 2, Nemetszrag
Munkahelyi es Privat: 891238485 / 535664646

Kereses irányitoszam alapjan (4400):
Nev: Kiss Kata (KisKata)
Cim: 4400 Nyiregyhaza, Kiss Pal utca 6, Magyarország
Munkahelyi es Privat: 123456789 / 987654321

Kereses ország alapjan (Nemetszrag):
Nev: Nagy Noemi (Noe)
Cim: 4025 Debrecen, Kossuth Lajos utca 2, Nemetszrag
Munkahelyi es Privat: 891238485 / 535664646

Kereses város alapjan (Mogyorod):
Nev: Kolos Vera (Vera)
Cim: 2234 Mogyorod, Tep utca 4, Magyarország
Munkahelyi es Privat: 06295760940 / 06448599209

Kereses nem letezo cim alapjan (Koranthi Frigyes utca 8):
Nincs talalat a(z) Koranthi Frigyes utca 8 cimre.
=====
```

SIKERES Telefonkonyv.Kereses_Cim_Alapjan <---

5.5 teszt_5 ()

Teszt 5: Profil keresése telefonszám alapján.

Ez a teszt a telefonkönyvben való keresést vizsgálja egy adott telefonszám alapján. A teszt során különböző telefonszámokkal rendelkező profilokat adunk hozzá a telefonkönyvhöz, majd megpróbálunk keresni egy adott számot (munkahelyi és privat). A keresett profil(ok) helyességét ellenőrizzük.

```
----> Telefonkonyv.Kereses_Tel_Alapjan
=====

5. teszt - Kereses (tel)
Kereses munkahelyi telefonszam alapjan (123456789):
Nev: Kiss Kata (KisKata)
Cim: 4400 Nyiregyhaza, Kiss Pal utca 6, Magyarország
Munkahelyi es Privat: 123456789 / 987654321

Kereses privat telefonszam alapjan (06448599209):
Nev: Kolos Vera (Vera)
Cim: 2234 Mogyorod, Tep utca 4, Magyarország
Munkahelyi es Privat: 06295760940 / 06448599209

Kereses nem letezo telefonszam alapjan (000000000):
Nincs talalat a(z) 000000000 telefonszagra.
=====
```

SIKERES Telefonkonyv.Kereses_Tel_Alapjan <---

5.6 Memóriakezelés tesztje

A memóriakezelés ellenőrzését a laborgyakorlatokon használt **MEMTRACE** modullal végeztem. Ehhez a programra rányomva a **Build options** lehetőségre, majd a **Debug**-nál **#defines**-ba beírtam a **MEMTRACE**-t. Memóriakezelési hibát nem tapasztaltam a futtatások során.

6. Mellékletek

6.1 string.h

```
00001 #ifndef STRING_H
00002 #define STRING_H
00003 #include <iostream>
00004
00005 //Felhasználtam a laboron alkotott részeket (forrásban megjelöltem)
00006
00007
00008
00009
00010 class String {
00011     char *pData;
00012     size_t len;
00013 public:
00014     void printDbg(const char *txt = "") const {
00015         std::cout << txt << "[" << len << "], "
00016             << (pData ? pData : "(NULL)") << '|' << std::endl;
00017     }
00018
00019     String() :pData(0), len(0) {}
00020
00021     size_t size() const { return len; }
00022
00023     const char* c_str() const { return pData ? pData : ""; }
00024
00025     ~String();
00026
00027     String(const char* str);
00028
00029     String(const String& rhs);
00030
00031     String& operator=(const String& rhs);
00032
00033     String operator+(const String& rhs) const;
00034
00035     bool operator==(const String& other) const;
00036
00037     friend std::istream& operator>>(std::istream& is, String& str);
00038
00039 };
00040
00041     std::ostream& operator<<(std::ostream& os, const String& str);
00042
00043 #endif
```


6.2 string.cpp

```
00001 #include <iostream>
00002 #include <cstring>
00003 #include "memtrace.h"
00004 #include "string.h"
00005
00008
00009 String::String(const char* str) {
00010     len = strlen(str);
00011     pData = new char[len + 1];
00012     strcpy(pData, str);
00013     pData[len]=0;
00014 }
00015
00017
00018 String::String(const String& rhs) {
00019     if (rhs.pData) {
00020         len = rhs.len;
00021         pData = new char[len + 1];
00022         strcpy(pData, rhs.pData);
00023     } else {
00024         len = 0;
00025         pData = nullptr;
00026     }
00027 }
00028
00029
00031
00032 String::~~String() {
00033     delete[] pData;
00034 }
00035
00037 String& String::operator=(const String& rhs) {
00038     if (this != &rhs) {
00039         delete[] pData;
00040
00041         if (rhs.pData) {
00042             len = rhs.len;
00043             pData = new char[len + 1];
00044             strcpy(pData, rhs.pData);
00045         } else {
00046             len = 0;
00047             pData = nullptr;
00048         }
00049     }
00050     return *this;
00051 }
00052
00053
00056
00057 String String::operator+(const String& rhs) const {
00058     size_t ujLen = len + rhs.len;
00059     char* ujData = new char[ujLen + 1];
00060     strcpy(ujData, pData);
00061     strcat(ujData, rhs.pData);
00062     String eredmeny(ujData);
00063     delete[] ujData;
00064     return eredmeny;
00065 }
00066
00067
00069 std::ostream& operator<<(std::ostream& os, const String& str) {
00070     os << str.c_str();
00071     return os;
00072 }
00073
00075 bool String::operator==(const String& other) const {
00076     if (len != other.len) return false;
00077     return strcmp(pData, other.pData) == 0;
00078 }
00079
```

```

00081 std::istream& operator>>(std::istream& is, String& str) {
00082     delete[] str.pData;
00083     str.pData = nullptr;
00084     str.len = 0;
00085
00086     char* temp = nullptr;
00087     char ch;
00088     size_t index = 0;
00089
00090     while (is.get(ch) && ch != '\n') {
00091         char* newTemp = new char[index + 2];
00092         if (temp) {
00093             memcpy(newTemp, temp, index);
00094             delete[] temp;
00095         }
00096         newTemp[index] = ch;
00097         newTemp[index + 1] = '\0';
00098         temp = newTemp;
00099         index++;
00100     }
00101     str.len = index;
00102     str.pData = temp;
00103     return is;

```

6.3 nev.h

```

00001 #ifndef NEV_H_INCLUDED
00002 #define NEV_H_INCLUDED
00003 #include <string>
00004 #include <iostream>
00005 #include "string.h"
00006
00010
00011 class Nev {
00012 private:
00013     String vnev;
00014     String knev;
00015     String bnev;
00016
00017 public:
00018     // Konstruktorkok
00020     Nev();
00025
00026     Nev(const String& vnev, const String& knev, const String& bnev);
00027
00031     Nev(const String& vnev, const String& knev);
00032
00033     // Getterek
00036     String getVnev() const;
00037
00040     String getKnev() const;
00041
00044     String getBnev() const;
00045
00048     String getTeljesNev() const;
00049
00052     Nev beolvas_nev();
00053 };
00054
00055     // Túlterheljük az << operátort
00060     std::ostream& operator<<(std::ostream& os, const Nev& rhs);
00061
00062
00063 #endif // NEV_H_INCLUDED

```

6.4 nev.cpp

```
00001 #include "Nev.h"
00002 #include "string.h"
00003 #include <iostream>
00004 #include <algorithm> // std::all_of
00005 #include <cctype> // ::isalpha
00006 #include "memtrace.h"
00007
00009 Nev::Nev() : vnev(""), knev(""), bnev("") {}
00010 Nev::Nev(const String& vnev, const String& knev, const String& bnev) : vnev(vnev),
knev(knev), bnev(bnev) {}
00011 Nev::Nev(const String& vnev, const String& knev) : vnev(vnev), knev(knev), bnev("") {}
00012
00014 String Nev::getVnev() const { return vnev; }
00015 String Nev::getKnev() const { return knev; }
00016 String Nev::getBnev() const { return bnev; }
00017 String Nev::getTeljesNev() const { return vnev + " " + knev; }
00018
00019
00021 Nev Nev::beolvas_nev() {
00022     String v, k, b;
00023     std::cin.clear();
00025     while (true) {
00026         std::cout << "Vezeteknev: ";
00027         std::cin >> v;
00028         if (v.size() > 0 && std::all_of(v.c_str(), v.c_str() + v.size(), ::isalpha)) {
00029             break;
00030         } else {
00031             std::cout << "Hiba: A vezeteknevnek legalabb 1 betubol kell allnia, es csak
betu lehet!" << std::endl;
00032         }
00033     }
00034
00036     while (true) {
00037         std::cout << "Keresztnev: ";
00038         std::cin >> k;
00039         if (k.size() > 0 && std::all_of(k.c_str(), k.c_str() + k.size(), ::isalpha)) {
00040             break;
00041         } else {
00042             std::cout << "Hiba: A keresztnevnek legalabb 1 betubol kell allnia, es csak
betu lehet!" << std::endl;
00043         }
00044     }
00045
00047     std::cout << "Becenev: ";
00048     std::cin >> b;
00049
00050     return Nev(v, k, b);
00051 }
00052
00053
00055 std::ostream& operator<<(std::ostream& os, const Nev& rhs) {
00056     os << "Nev: " << rhs.getTeljesNev();
00057
00059     if (strlen(rhs.getBnev().c_str()) > 0) {
00060         os << " (" << rhs.getBnev() << ")";
00061     }
00062
00063     return os;
00064 }
```

6.5 cim.h

```
00001 #ifndef CIM_H_INCLUDED
00002 #define CIM_H_INCLUDED
00003 #include <iostream>
00004 #include "string.h"
00005 #include <iostream>
00006
00009
00010 class Cim {
00011 private:
00012     String orszag;
00013     String irányitoszam;
00014     String varos;
00015     String utca;
00016
00017 public:
00018     // Konstruktorkok
00020     Cim();
00021
00027     Cim(const String& orszag, const String& irányitoszam, const String& varos, const
String& utca);
00028
00029     // Getterek
00030
00033     String getOrszag() const;
00034
00037     String getIranyito() const;
00038
00041     String getVaros() const;
00042
00045     String getUtca() const;
00046
00047     // Cím beolvasása
00050     Cim beolvas_cim();
00051
00052
00053 };
00054
00055     // Túlterheljük az << operátort
00060     std::ostream& operator<<(std::ostream& os, const Cim& rhs);
00061
00062 #endif // CIM_H_INCLUDED
```

6.6 cim.cpp

```
00001
00002 #include "Cim.h"
00003 #include "string.h"
00004 #include "memtrace.h"
00005 #include <cstring> // A C-sztringek kezel  s  hez
00006 #include <algorithm> // Az std::all_of haszn  lat  hoz
00007
00008 Cim::Cim() : orszag(""), irányitoszam(""), varos(""), utca("") {}
00009
00010
00011 Cim::Cim(const String& orszag, const String& irányitoszam, const String& varos, const String& utca)
00012     : orszag(orszag), irányitoszam(írányitoszam), varos(varos), utca(utca) {}
00013
00014
00015 String Cim::getOrszag() const { return orszag; }
00016 String Cim::getÍrányito() const { return irányitoszam; }
00017 String Cim::getVaros() const { return varos; }
00018 String Cim::getUtca() const { return utca; }
00019
00020
00021 Cim Cim::beolvas_cim() {
00022     String orsz, irsz, var, utc;
00023
00024     while (true) {
00025         std::cout << "Orszag: ";
00026         std::cin >> orsz;
00027         if (orsz.size() > 0 && std::all_of(orsz.c_str(), orsz.c_str() + orsz.size(), ::isalpha)) {
00028             break;
00029         } else {
00030             std::cout << "Hiba: Az orszag neve nem lehet ures, es nem tartalmazhat szamokat,
00031 szokozokat." << std::endl;
00032         }
00033     }
00034
00035     while (true) {
00036         std::cout << "Írányitoszam (4 karakter, csak szamok): ";
00037         std::cin >> irsz;
00038         if (irsz.size() == 4 && std::all_of(irsz.c_str(), irsz.c_str() + 4, ::isdigit)) {
00039             break;
00040         } else {
00041             std::cout << "Hiba: Az irányitoszamn  k 4 sz  mjegy  bol kell allnia." << std::endl;
00042         }
00043     }
00044
00045     while (true) {
00046         std::cout << "Varos: ";
00047         std::cin >> var;
00048         if (var.size() > 0 && std::all_of(var.c_str(), var.c_str() + var.size(), ::isalpha)) {
00049             break;
00050         } else {
00051             std::cout << "Hiba: A varos neve nem lehet ures, es nem tartalmazhat szamokat,
00052 szokozokat." << std::endl;
00053         }
00054     }
00055
00056     while (true) {
00057         std::cout << "Utca: ";
00058         std::cin >> utc;
00059         if (utc.size() > 0) {
00060             break;
00061         } else {
00062             std::cout << "Hiba: Az utca neve nem lehet ures" << std::endl;
00063         }
00064     }
00065
00066     return Cim(orsz, irsz, var, utc);
00067 }
00068
00069
00070 std::ostream& operator<<(std::ostream& os, const Cim& rhs) {
00071     os << "Cim: " << rhs.getÍrányito() << " " << rhs.getVaros() << ", " << rhs.getUtca() << ", " <<
00072     rhs.getOrszag();
00073     return os;
00074 }
```

6.7 tel.h

```
00001 #ifndef TEL_H_INCLUDED
00002 #define TEL_H_INCLUDED
00003 #include "string.h"
00004 #include <iostream>
00005
00006
00007
00008
00009 class Tel {
00010 private:
00011     String mszam;
00012     String pszam;
00013
00014 public:
00015     // Konstruktorok
00016     Tel();
00017
00018
00019
00020
00021
00022     Tel(const String& mszam, const String& pszam);
00023
00024     // Getterek
00025
00026
00027
00028     String getMszam() const;
00029
00030
00031
00032     String getPszam() const;
00033
00034
00035     // Telefonszám beolvasása
00036     Tel beolvas_tel();
00037
00038
00039
00040
00041 };
00042
00043     // Túlterheljük az << operátort
00044     std::ostream& operator<<(std::ostream& os, const Tel& rhs);
00045
00046
00047
00048
00049
00050
00051 #endif // TEL_H_INCLUDED
```

6.8 tel.cpp

```
00001 #include "Tel.h"
00002 #include "string.h"
00003 #include "memtrace.h"
00004 #include <iostream>
00005 #include <algorithm>
00006 #include <cctype>
00007
00008
00010 Tel::Tel() : mszam(""), pszam("") {}
00011
00013 Tel::Tel(const String& mszam, const String& pszam)
00014     : mszam(mszam), pszam(pszam) {}
00015
00017 String Tel::getMszam() const { return mszam; }
00018 String Tel::getPszam() const { return pszam; }
00019
00020
00022 Tel Tel::beolvas_tel() {
00023     String msz, psz;
00024
00026     while (true) {
00027         std::cout << "Munkahelyi szam (11 karakter, csak szamok): ";
00028         std::cin >> msz;
00029
00031         if (msz.size() == 11 && std::all_of(msz.c_str(), msz.c_str() + 11, ::isdigit)) {
00032             break;
00033         } else {
00034             std::cout << "Hiba: A munkahelyi szamnak 11 szamjegyből kell allnia." <<
std::endl;
00035         }
00036     }
00037
00039     while (true) {
00040         std::cout << "Privat szam (11 karakter, csak szamok): ";
00041         std::cin >> psz;
00042
00044         if (psz.size() == 11 && std::all_of(psz.c_str(), psz.c_str() + 11, ::isdigit)) {
00045             break;
00046         } else {
00047             std::cout << "Hiba: A privat szamnak 11 szamjegyből kell allnia." <<
std::endl;
00048         }
00049     }
00050
00051     return Tel(msz, psz);
00052 }
00053
00054
00056 std::ostream& operator<<(std::ostream& os, const Tel& rhs) {
00057     os << "Munkahelyi es Privat: " << rhs.getMszam() << " / " << rhs.getPszam();
00058     return os;
00059 }
```

6.9 profil.h

```
00001 #ifndef PROFIL_H_INCLUDED
00002 #define PROFIL_H_INCLUDED
00003 #include "Nev.h"
00004 #include "Cim.h"
00005 #include "Tel.h"
00006 #include <iostream>
00007
00010
00011
00012 class Profil {
00013 private:
00014     Nev nev;
00015     Cim cim;
00016     Tel tel;
00017
00018 public:
00019     // Konstrutorok
00021     Profil();
00022
00027     Profil(const Nev& nev, const Cim& cim, const Tel& tel);
00028
00029
00030     // Getterek
00031
00034     Nev getNev() const;
00035
00038     Cim getCim() const;
00039
00042     Tel getTel() const;
00043
00044
00045 };
00046
00047     // Túlterheljük az << operátort
00052     std::ostream& operator<<(std::ostream& os, const Profil& rhs);
00053
00054 #endif // PROFIL_H_INCLUDED
```

6.10 profil.cpp

```
00001
00002 #include "Profil.h"
00003 #include "memtrace.h"
00004
00006 Profil::Profil() : nev(), cim(), tel() {}
00007
00009 Profil::Profil(const Nev& nev, const Cim& cim, const Tel& tel)
00010     : nev(nev), cim(cim), tel(tel) {}
00011
00012
00014 Nev Profil::getNev() const { return nev; }
00015 Cim Profil::getCim() const { return cim; }
00016 Tel Profil::getTel() const { return tel; }
00017
00018
00020 std::ostream& operator<<(std::ostream& os, const Profil& rhs) {
00021     os << rhs.getNev() << "\n" << rhs.getCim() << "\n" << rhs.getTel() << std::endl;
00022     return os;
00023 }
```


6.11 telefonkonyv.h

```
00001 #ifndef TELEFONKONYV_H_INCLUDED
00002 #define TELEFONKONYV_H_INCLUDED
00003 #include "Profil.h"
00004 #include <iostream>
00005
00009
00010 class Telefonkonyv {
00011 private:
00012     Profil* profiltar;
00013     size_t profilSzam;
00014     size_t kapacitas;
00015
00016     // Kapacitás növelése
00019     void noveldKapacitas();
00020
00021 public:
00023     Telefonkonyv(); // Konstruktor
00024
00025     //Destruktor
00027     ~Telefonkonyv();
00028
00029     // Adatok felvétele
00030     void addProfil(const Profil& profil);
00031
00032     // Listázás
00034     void listaz() const;
00035
00036     // Adatok törlése
00040     void torol(const String& vnev, const String& knev);
00041
00042     // Egyszerű keresés
00046     bool keresNev(const String& nev) const;
00047
00048     // Keresés cím alapján
00052     bool keresCim(const String& cim) const;
00053
00054     // Keresés telefonszám alapján
00058     bool keresTel(const String& tel) const;
00059
00060     // Profilok számának lekérdezése
00063     size_t getProfilSzam() const;
00064
00065 };
00066
00067
00068 #endif // TELEFONKONYV_H_INCLUDED
```

6.12 telefonkonyv.cpp

```
00001 #include "Telefonkonyv.h"
00002 #include "memtrace.h"
00003
00005 Telefonkonyv::Telefonkonyv() : profilSzam(0), kapacitas(1) {
00006     profiltar = new Profil[kapacitas];
00007 }
00008
00010 Telefonkonyv::~Telefonkonyv() {
00011     delete[] profiltar;
00012 }
00013
00015 void Telefonkonyv::noveldKapacitas() {
00016     kapacitas++;
00017     Profil* ujTar = new Profil[kapacitas];
00018     for (size_t i = 0; i < profilSzam; i++) {
00019         ujTar[i] = profiltar[i];
00020     }
00021     delete[] profiltar;
00022     profiltar = ujTar;
00023 }
00024
00026 void Telefonkonyv::addProfil(const Profil& profil) {
00027     if (profilSzam >= kapacitas) {
00028         noveldKapacitas();
00029     }
00030     profiltar[profilSzam] = profil;
00031     profilSzam++;
00032 }
00033
00035 void Telefonkonyv::listaz() const {
00036     for (size_t i = 0; i < profilSzam; i++) {
00037         std::cout << profiltar[i] << std::endl;
00038     }
00039 }
00040
00042 void Telefonkonyv::torol(const String& vnev, const String& knev) {
00043     bool talalt = false;
00044     for (size_t i = 0; i < profilSzam; i++) {
00045         if (profiltar[i].getNev().getVnev() == vnev && profiltar[i].getNev().getKnev()
00046             == knev) {
00047             for (size_t j = i; j < profilSzam - 1; j++) {
00048                 profiltar[j] = profiltar[j + 1];
00049             }
00050             profilSzam--;
00051             std::cout << "Profil torolve: " << vnev << " " << knev << std::endl;
00052             talalt = true;
00053         }
00054     }
00055     if (!talalt) {
00056         std::cout << "Profil nem talalhato: " << vnev << " " << knev << std::endl;
00057     }
00058 }
00059
00061 bool Telefonkonyv::keresNev(const String& nev) const {
00062     bool talalat = false;
00063     for (size_t i = 0; i < profilSzam; i++) {
00064         if (profiltar[i].getNev().getVnev() == nev ||
00065             profiltar[i].getNev().getKnev() == nev ||
00066             profiltar[i].getNev().getTeljesNev() == nev ||
00067             profiltar[i].getNev().getBnev() == nev) {
00068             std::cout << profiltar[i] << std::endl;
00069             talalat = true;
00070         }
00071     }
00072     if (!talalat) {
00073         std::cout << "Nincs talalat a(z) " << nev << " nevre." << std::endl;
00074     }
00075     return talalat;
00076 }
00077
00079 bool Telefonkonyv::keresCim(const String& cim) const {
00080     bool talalat = false;
```

```

00083     for (size_t i = 0; i < profilSzam; i++) {
00084         if (profiltar[i].getCim().getUtca() == cim ||
00085             profiltar[i].getCim().getVaros() == cim ||
00086             profiltar[i].getCim().getOrszag() == cim ||
00087             profiltar[i].getCim().getIranyito() == cim) {
00088             std::cout << profiltar[i] << std::endl;
00089             talalat = true;
00090         }
00091     }
00092     if (!talalat) {
00093         std::cout << "Nincs talalat a(z) " << cim << " cimre." << std::endl;
00094     }
00095     return talalat;
00096 }
00097
00099 bool Telefonkonyv::keresTel(const String& tel) const {
00100     bool talalat = false;
00101     for (size_t i = 0; i < profilSzam; i++) {
00102         if (profiltar[i].getTel().getMszam() == tel ||
00103             profiltar[i].getTel().getPszam() == tel) {
00104             std::cout << profiltar[i] << std::endl;
00105             talalat = true;
00106         }
00107     }
00108     if (!talalat) {
00109         std::cout << "Nincs talalat a(z) " << tel << " telefonszamra." << std::endl;
00110     }
00111     return talalat;
00112 }
00113
00115 size_t Telefonkonyv::getProfilSzam() const {
00116     return profilSzam;
00117 }

```

6.13 teszt.h

```

00001 #ifndef TESZT_H_INCLUDED
00002 #define TESZT_H_INCLUDED
00003
00013
00014 void kiir(int hossz, char c);
00015
00024
00025 void clearScreen();
00026
00034
00035 void waitForEnter();
00036
00037
00046
00047 void teszt_1();
00048
00058
00059 void teszt_2();
00060
00069
00070 void teszt_3();
00071
00080
00081 void teszt_4();
00082
00092
00093 void teszt_5();
00094
00095
00096 #endif // TESZT_H_INCLUDED

```

6.14 teszt.cpp

```
00001 #include <iostream>
00002 #include "nev.h"
00003 #include "cim.h"
00004 #include "tel.h"
00005 #include "profil.h"
00006 #include "telefonkonyv.h"
00007 #include "string.h"
00008 #include "memtrace.h"
00009 #include "gtest_lite.h"
00010
00011
00012 void kiir(int hossz, char c) {
00013     for (int i = 0; i < hossz; i++) {
00014         std::cout << c;
00015     }
00016     std::cout << std::endl;
00017 }
00018
00019 void clearScreen() {
00020     #ifdef _WIN32
00021         system("cls");
00022     #else
00023         system("clear");
00024     #endif
00025 }
00026
00027 void waitForEnter() {
00028     std::cout << "Nyomj meg egy ENTER-t a folytatashoz..." << std::endl;
00029
00030     fflush(stdin);
00031     getchar();
00032 }
00033 }
00034
00035
00036 void teszt_1() {
00037     TEST(Telefonkonyv, AdatokFelvetele) {
00038         kiir(50, '=');
00039         std::cout << "\n1. teszt - Adatok felvetele kodban " << std::endl;
00040
00041         Telefonkonyv t1;
00042
00043         Profil p1(Nev("Kiss", "Kata", "KisKata"), Cim("Magyarország", "4400",
00044 "Nyiregyhaza", "Kiss Pal utca 6"), Tel("12345678910", "10987654321"));
00045         Profil p2(Nev("Nagy", "Noemi", "Noe"), Cim("Magyarország", "4025", "Debrecen",
00046 "Kossuth Lajos utca 2"), Tel("89123848510", "53566464610"));
00047         Profil p3(Nev("Kolos", "Vera", "Vera"), Cim("Magyarország", "2234", "Mogyorod",
00048 "Tep utca 4"), Tel("06295760940", "06448599209"));
00049
00050         t1.addProfil(p1);
00051         t1.addProfil(p2);
00052         t1.addProfil(p3);
00053
00054         EXPECT_EQ(static_cast<size_t>(3), t1.getProfilSzam()) << "A profilok szama nem
00055 megfelelo!" << std::endl;
00056
00057         std::cout << "\nProfilok listazasa:\n";
00058         t1.listaz();
00059
00060         kiir(50, '=');
00061         waitForEnter();
00062         clearScreen();
00063
00064         kiir(50, '=');
00065         std::cout << "\nProfil torlese (Kiss Kata):\n";
00066         t1.torol("Kiss", "Kata");
00067
00068         EXPECT_EQ(static_cast<size_t>(2), t1.getProfilSzam()) << "A profilok szama nem
00069 megfelelo!" << std::endl;
00070
00071         std::cout << "\nProfilok listazasa:\n";
00072         t1.listaz();
00073
00074
00075 }
```

```

00076
00077     kiir(50, '=');
00078     std::cout << "Kilepes az elso tesztbol! (ENTER) \n";
00079     waitForEnter();
00080     clearScreen();
00081 } END
00082 }
00083
00084 void teszt_2() {
00085     TEST(Telefonkonyv, Adatok_Felvetele_Szabvanyos_Bemenetrol) {
00086         kiir(50, '=');
00087         std::cout << "\n2. teszt - Adatok felvetele szabvanyos bemenetrol" << std::endl;
00088
00089         Telefonkonyv t1;
00090         int profilSzam;
00091
00092         do {
00093             std::cout << "Hany profilt szeretnel felvenni? ";
00094             std::cin >> profilSzam;
00095
00096             if (profilSzam <= 0) {
00097                 std::cout << "Kerlek, adj meg egy pozitiv szamot!" << std::endl;
00098             }
00099             std::cin.ignore();
00100         } while (profilSzam <= 0);
00101
00102         for (int i = 0; i < profilSzam; i++) {
00103             std::cout << "\nProfil " << (i + 1) << ". felvetele: " << std::endl;
00104
00105             Nev n = n.beolvas_nev();
00106             Cim c = c.beolvas_cim();
00107             Tel t = t.beolvas_tel();
00108
00109             Profil profil(n, c, t);
00110
00111             t1.addProfil(profil);
00112         }
00113
00114         EXPECT_EQ(static_cast<size_t>(profilSzam), t1.getProfilSzam()) << "A profilok
szama nem megfelelo!" << std::endl;
00115
00116         std::cout << "\nProfilok listazasa:\n";
00117         t1.listaz();
00118
00119         kiir(50, '=');
00120         std::cout << "Kilepes a masodik tesztbol! (ENTER)\n";
00121         waitForEnter();
00122         clearScreen();
00123     } END
00124 }
00125
00126 void teszt_3() {
00127     TEST(Telefonkonyv, Kereses_Nev_Alapjan) {
00128         kiir(50, '=');
00129         std::cout << "\n3. teszt - Kereses (nev)" << std::endl;
00130
00131         Telefonkonyv t1;
00132         Profil p1(Nev("Kiss", "Kata", "KisKata"), Cim("Magyarország", "4400",
"Nyiregyhaza", "Kiss Pal utca 6"), Tel("12345678910", "98765432110"));
00133         Profil p2(Nev("Nagy", "Noemi", "Noe"), Cim("Nemetsország", "4025", "Debrecen",
"Kossuth Lajos utca 2"), Tel("89123848510", "53566464610"));
00134         Profil p3(Nev("Kolos", "Vera", "Vera"), Cim("Magyarország", "2234", "Mogyorod",
"Tep utca 4"), Tel("06295760940", "06448599209"));
00135
00136         t1.addProfil(p1);
00137         t1.addProfil(p2);
00138         t1.addProfil(p3);
00139
00140         std::cout << "Kereses teljes nev alapjan (Kiss Kata):\n";
00141         EXPECT_TRUE(t1.keresNev("Kiss Kata")) << "A kereses nem talalta meg a teljes
nevet!";
00142
00143         std::cout << "Kereses vezeteknev alapjan (Kiss):\n";
00144         EXPECT_TRUE(t1.keresNev("Kiss")) << "A kereses nem talalta meg a vezeteknevet!";
00145     }
00146 }

```

```

00160
00162         std::cout << "Kereses keresztnév alapján (Noemi):\n";
00163         EXPECT_TRUE(t1.keresNev("Noemi")) << "A kereses nem talalta meg a
keresztnévet!";
00164
00166         std::cout << "Kereses becenev alapján (Vera):\n";
00167         EXPECT_TRUE(t1.keresNev("Vera")) << "A kereses nem talalta meg a becenevet!";
00168
00170         std::cout << "Kereses nem letezo nev alapján (Nem letezo Nev):\n";
00171         EXPECT_FALSE(t1.keresNev("Anna")) << "A kereses nem vart talalatot adott a nem
letezo nev alapján!";
00172
00175         std::cout << "\nProfil torlese (Kiss Kata):\n";
00176         t1.torol("Kiss", "Kata");
00177         std::cout << "Kereses nev alapján (Kiss) torles utan:\n";
00178         EXPECT_FALSE(t1.keresNev("Kiss")) << "A kereses nem vart talalatot adott a
torles utan!";
00179
00180         kiir(50, '=');
00181         std::cout << "Kilepes a harmadik tesztbol! (ENTER)\n";
00182         waitForEnter();
00183         clearScreen();
00184     } END
00185 }
00186
00187 void teszt_4() {
00188     TEST(Telefonkonyv, Kereses_Cim_Alapjan) {
00189         kiir(50, '=');
00190         std::cout << "\n4. teszt - Kereses (cim)" << std::endl;
00191
00192         Telefonkonyv t1;
00194         Profil p1(Nev("Kiss", "Kata", "KisKata"), Cim("Magyarország", "4400",
"Nyiregyhaza", "Kiss Pal utca 6"), Tel("12345678910", "98765432110"));
00195         Profil p2(Nev("Nagy", "Noemi", "Noe"), Cim("Nemország", "4025", "Debrecen",
"Kossuth Lajos utca 2"), Tel("89123848510", "53566464610"));
00196         Profil p3(Nev("Kolos", "Vera", "Vera"), Cim("Magyarország", "2234", "Mogyorod",
"Tep utca 4"), Tel("06295760940", "06448599209"));
00197
00198         t1.addProfil(p1);
00199         t1.addProfil(p2);
00200         t1.addProfil(p3);
00201
00203
00205         std::cout << "Kereses utca alapján (Kossuth Lajos utca 2):\n";
00206         EXPECT_TRUE(t1.keresCim("Kossuth Lajos utca 2")) << "A kereses nem talalta meg a
cimet!";
00207
00209         std::cout << "Kereses irányitoszam alapján (4400):\n";
00210         EXPECT_TRUE(t1.keresCim("4400")) << "A kereses nem talalta meg az
irányitoszamat!";
00211
00213         std::cout << "Kereses ország alapján (Nemország):\n";
00214         EXPECT_TRUE(t1.keresCim("Nemország")) << "A kereses nem talalta meg az
orszagot!";
00215
00217         std::cout << "Kereses város alapján (Mogyorod):\n";
00218         EXPECT_TRUE(t1.keresCim("Mogyorod")) << "A kereses nem talalta meg a varost!";
00219
00221         std::cout << "Kereses nem letezo cim alapján (Koranth Frigyes utca 8):\n";
00222         EXPECT_FALSE(t1.keresCim("Koranth Frigyes utca 8")) << "A kereses nem vart
talalatot adott a nem letezo cim alapján!";
00223
00224         kiir(50, '=');
00225         std::cout << "Kilepes a negyedik tesztbol! (ENTER)\n";
00226         waitForEnter();
00227         clearScreen();
00228     } END
00229 }
00230
00231 void teszt_5() {
00232     TEST(Telefonkonyv, Kereses_Tel_Alapjan) {
00233         kiir(50, '=');
00234         std::cout << "\n5. teszt - Kereses (tel)" << std::endl;
00235
00236         Telefonkonyv t1;
00238         Profil p1(Nev("Kiss", "Kata", "KisKata"), Cim("Magyarország", "4400",
"Nyiregyhaza", "Kiss Pal utca 6"), Tel("12345678910", "98765432110"));

```

```

00239     Profil p2(Nev("Nagy", "Noemi", "Noe"), Cim("Nemetország", "4025", "Debrecen",
00240 "Kossuth Lajos utca 2"), Tel("89123848510", "53566464610"));
00241     Profil p3(Nev("Kolos", "Vera", "Vera"), Cim("Magyarország", "2234", "Mogyorod",
00242 "Tep utca 4"), Tel("06295760940", "06448599209"));
00243     t1.addProfil(p1);
00244     t1.addProfil(p2);
00245     t1.addProfil(p3);
00246
00247     std::cout << "Kereses munkahelyi telefonszam alapján (12345678910):\n";
00248     EXPECT_TRUE(t1.keresTel("12345678910")) << "A kereses nem talalta meg a
00249 munkahelyi telefonszamat!";
00250
00251     std::cout << "Kereses privat telefonszam alapján (06448599209):\n";
00252     EXPECT_TRUE(t1.keresTel("06448599209")) << "A kereses nem talalta meg a privat
00253 telefonszamat!";
00254
00255     std::cout << "Kereses nem letezo telefonszam alapján (00000000000):\n";
00256     EXPECT_FALSE(t1.keresTel("00000000000")) << "A kereses nem vart talalatot adott
00257 a nem letezo telefonszam alapján!";
00258
00259     kiir(50, '=');
00260
00261     std::cout << "Kilepes az otodik tesztbol! (ENTER)\n";
00262     waitEnter();
00263     clearScreen();
00264 } END
00265 }
00266 }

```

6.15 menu.h

```

00001 #ifndef MENUK_H_INCLUDED
00002 #define MENUK_H_INCLUDED
00003 void teszteket();
00004
00005 void elso_menu(Telefonkonyv& t1);
00006
00007 void harmadik_menu(Telefonkonyv& t1);
00008
00009 void negyedik_menu(Telefonkonyv& t1);
00010
00011 #endif // MENUK_H_INCLUDED

```

6.16 menu.cpp

```

00001 #include "teszt.h"
00002 #include <iostream>
00003 #include "telefonkonyv.h"
00004
00005 void teszteket() {
00006     int teszt;
00007     do {
00008         kiir(50, '=');
00009         std::cout << "Valassz egy tesztet:\n \n";
00010         std::cout << "(1) Adatok felvetele kodban\n";
00011         std::cout << "(2) Adatok felvetele szabvanyos bemenetrol\n";
00012         std::cout << "(3) Kereses nev alapján\n";
00013         std::cout << "(4) Kereses cim alapján\n";
00014         std::cout << "(5) Kereses telefonszam alapján\n";
00015         std::cout << "(0) Vissza a fomenube\n\n";
00016         kiir(50, '=');
00017         std::cout << "Valasztas: ";
00018         std::cin >> teszt;
00019         clearScreen();
00020         switch (teszt) {

```

```

00021         case 1:
00022             teszt_1();
00023             break;
00024         case 2:
00025             teszt_2();
00026             break;
00027         case 3:
00028             teszt_3();
00029             break;
00030         case 4:
00031             teszt_4();
00032             break;
00033         case 5:
00034             teszt_5();
00035             break;
00036         case 0:
00037             return;
00038         default:
00039             std::cout << "Ervenytelen valasz, probald ujra.\n";
00040             break;
00041     }
00042     } while (teszt!= 0);
00043 }
00044
00045 void elso_menu(Telefonkonyv& t1) {
00046
00047     char megerosites;
00048     do {
00049         std::cout << "Biztosan folytatni szeretned a profil hozzaadasat? (Igen = 'y',
00050 Kilepes = '0'): ";
00051         std::cin >> megerosites;
00052
00053         if (megerosites == '0') {
00054             clearScreen();
00055             return;
00056         }
00057
00058         if (megerosites != 'y') {
00059             std::cout << "Ervenytelen valasz, kerlek probald ujra!\n";
00060         }
00061
00062     } while (megerosites != 'y' && megerosites != '0');
00063
00064     std::cin.ignore();
00065     Nev n = n.beolvas_nev();
00066     Cim c = c.beolvas_cim();
00067     Tel t = t.beolvas_tel();
00068     Profil p(n, c, t);
00069     t1.addProfil(p);
00070     clearScreen();
00071     std::cout << "Profil hozzaadva!" << std::endl;
00072 }
00073
00074 void harmadik_menu(Telefonkonyv& t1) {
00075     std::cin.ignore();
00076     String v,k;
00077     t1.listaz();
00078     kiir(50, '=');
00079     std::cout << "Add meg a torolni kivant profil vezeteknevet es keresztnevet
00080 (ugyanolyan formaban):\n";
00081     std::cout << "(Kilepes = 0) Vezeteknev: ";
00082     std::cin >> v;
00083     if (v == "0") {
00084         clearScreen();
00085         return;
00086     }
00087     std::cout << "Keresztnev: ";
00088     std::cin >> k;
00089     clearScreen();
00090     t1.torol(v, k);
00091 }
00092
00093 void negyedik_menu(Telefonkonyv& t1) {
00094     int valasz;
00095
00096     String keresett;

```



```

00099     do{
00100         kiir(50, '=');
00101         std::cout << "Kereses alapjan:\n\n";
00102         std::cout << "(1) Nev\n";
00103         std::cout << "(2) Cim\n";
00104         std::cout << "(3) Telefonszam\n";
00105         std::cout << "(0) Kilepes\n\n";
00106         kiir(50, '=');
00107         std::cout << "Valasztas: ";
00108         std::cin >> valasz;
00109         std::cin.ignore();
00110         clearScreen();
00111
00112         switch (valasz) {
00113             case 1:
00114                 std::cout << "Add meg a keresett nevet: ";
00115                 std::cin >> keresett;
00116                 t1.keresNev(keresett);
00117                 break;
00118             case 2:
00119                 std::cout << "Add meg a keresett cimet: ";
00120                 std::cin >> keresett;
00121                 t1.keresCim(keresett);
00122                 break;
00123             case 3:
00124                 std::cout << "Add meg a keresett telefonszamat: ";
00125                 std::cin >> keresett;
00126                 t1.keresTel(keresett);
00127                 break;
00128             case 0:
00129                 return;
00130             default:
00131                 std::cout << "Ervenytelen valasz, kerlek probald ujra.\n";
00132                 break;
00133         }
00134     }while (valasz != 0);
00135 }

```

6.17 main.cpp

```

00001
00052
00053
00054 #include <iostream>
00055 #include "nev.h"
00056 #include "cim.h"
00057 #include "tel.h"
00058 #include "profil.h"
00059 #include "telefonkonyv.h"
00060 #include "string.h"
00061 #include "memtrace.h"
00062 #include "gtest_lite.h"
00063 #include "teszt.h"
00064 #include "menuk.h"
00065
00066 //Forr sok
00067 //https://en.cppreference.com/w/cpp/language/string_literal#Raw_string_literals --
>Grafika
00068 //https://www.asciart.eu/text-to-ascii-art --->Grafika
00069 //https://infocpp.iit.bme.hu/tananyag
00070 //https://infocpp.iit.bme.hu/hf_hez
00071 //https://en.cppreference.com/w/cpp/language/operators
00072 //https://en.cppreference.com/w/cpp/algorithm/all_any_none_of --> std::all_of
00073 //https://en.cppreference.com/w/cpp/string/byte/isdigit --> std::isdigit
00074 //https://en.wikipedia.org/wiki/Conio.h
00075 //https://cplusplus.com/reference/
00076 //Felhaszn lt elemek: memtrace.h, memtrace.cpp, gtest_lite.h, string.h, string.cpp
00077 //https://bmeaut.github.io/snippets/snippets/0105_Doxygen/
00078
00079
00080
00081

```

[illegible]

7. Források

Raw string literals – cppreference	Grafika
ASCII Art Generator	Grafika
IIT BME C++ tananyag	C++ programozási alapok
IIT BME házi feladatok	C++ házi feladatok
C++ operátorok - cppreference	Operátorok ismertetése
std::all_of - cppreference	Az all_of algoritmus használata
std::isdigit – cppreference	Az isdigit függvény használata
Conio.h Wikipedia	A conio.h könyvtár ismertetése
Cplusplus.com Reference	A C++ referencia oldal
BME Aut - Snippets Doxygen Documentation	Doxygen dokumentáció

https://en.cppreference.com/w/cpp/language/string_literal#Raw_string_literals

<https://www.asciiart.eu/text-to-ascii-art>

<https://infocpp.iit.bme.hu/tananyag>

https://infocpp.iit.bme.hu/hf_hez

<https://en.cppreference.com/w/cpp/language/operators>

https://en.cppreference.com/w/cpp/algorithm/all_any_none_of

<https://en.cppreference.com/w/cpp/string/byte/isdigit>

<https://en.wikipedia.org/wiki/Conio.h>

<https://cplusplus.com/reference/>

Felhasznált elemek: memtrace.h, memtrace.cpp, gtest_lite.h, string.h, string.cpp

https://bmeaut.github.io/snippets/snippets/0105_Doxygen/